HOSPITAL MANAGEMENT SYSTEM

QA AUDIT REPORT: **PASS (100%)**

Date: October 28, 2026

Document: QA-AUDIT-2026-PDF

Master Test Cases & Formal Defect / Bug Report

Complete enterprise quality audit verifying functional endpoints, authentication lifecycle, role-based boundaries, cross-tenant IDOR protection, billing equations, inpatient census, and defect remediation.

TOTAL TEST CASES	TEST PASS RATE	REGRESSION TESTS	FRONTEND TESTS	DEFECTS RESOLVED	RELEASE STATUS
26 Formal live audits	**100%** 26 / 26 Passed	**198** Django Unit Suite OK	**10** Vitest Integration OK	**1 / 1** BUG-001 Closed	**APPROVED** Ready for Production

1. Master Test Cases Inventory & Verification Log

Test ID	Module	Test Scenario	Execution Steps & Payload	Expected & Observed Result	Status	Severity
TC-ENV-01	Environment	System Health Check	`GET /api/v1/health/` `No auth header required`	HTTP 200 OK. Services healthy (database: healthy).	**PASS**	**Critical**
TC-ENV-02	Environment	Swagger API Docs	`GET /api/docs/` `OpenAPI 3.0 schema inspection`	HTTP 200 OK. Swagger schema successfully generated.	**PASS**	**Medium**
TC-ENV-03	Environment	Frontend Web App	`GET http://127.0.0.1:3000/` `Vite dev server bootstrap`	HTTP 200 OK. React 19 single-page application loaded.	**PASS**	**Critical**
TC-AUTH-01	Authentication	Valid Admin Login	`POST /api/v1/accounts/login/` `{'username': 'admin', 'password': '***'}`	HTTP 200 OK. Access and refresh JWT tokens generated.	**PASS**	**Critical**
TC-AUTH-02	Authentication	Incorrect Password	`POST /api/v1/accounts/login/` `{'username': 'admin', 'password': 'wrong'}`	HTTP 401 Unauthorized. Access denied with proper error.	**PASS**	**High**
TC-AUTH-03	Authentication	Non-Existent User	`POST /api/v1/accounts/login/` `{'username': 'ghost_user_9999', ...}`	HTTP 401 Unauthorized. User absence securely handled.	**PASS**	**High**
TC-AUTH-04	Authentication	Empty Credentials	`POST /api/v1/accounts/login/` `{'username': '', 'password': ''}`	HTTP 400 Bad Request. Field validation rules active.	**PASS**	**Medium**
TC-AUTH-05	Authentication	Token Refresh	`POST /api/v1/accounts/token/refresh/` `{'refresh': ''}`	HTTP 200 OK. New access token minted without re-login.	**PASS**	**High**
TC-AUTH-06	Authentication	Tampered Token	`POST /api/v1/accounts/token/refresh/` `{'refresh': 'forged.signature.token'}`	HTTP 401 Unauthorized. Cryptographic signature verified.	**PASS**	**High**
TC-AUTH-07	Authentication	Profile Unauthenticated	`GET /api/v1/accounts/profile/` `No Bearer token present`	HTTP 401 Unauthorized. Protected route blocked access.	**PASS**	**Critical**
TC-AUTH-08	Authentication	Profile Authenticated	`GET /api/v1/accounts/profile/` `Authorization: Bearer`	HTTP 200 OK. Profile retrieved (username, role: SUPER_ADMIN).	**PASS**	**High**

Test ID	Module	Test Scenario	Execution Steps & Payload	Expected & Observed Result	Status	Severity
TC-RBAC-01	RBAC / Staff	Patient Accessing Staff	`GET /api/v1/staff/` `Authorization: Bearer`	Cycle 1: 200 OK (Defect BUG-001). Cycle 2: HTTP 403 Forbidden [VERIFIED FIXED].	**PASS (Fixed)**	**High**
TC-RBAC-02	RBAC / Wards	Patient Accessing Wards	`GET /api/v1/inpatient/wards/` `Authorization: Bearer`	HTTP 403 Forbidden. Clinical wards restricted from patient.	**PASS**	**High**
TC-RBAC-03	RBAC / Clinical	Doctor Patient Access	`GET /api/v1/patients/` `Authorization: Bearer`	HTTP 200 OK. Doctor Dr. Sharma authorized to view roster.	**PASS**	**High**
TC-RBAC-04	RBAC / Pharmacy	Pharmacist Drug Access	`GET /api/v1/pharmacy/medicines/` `Authorization: Bearer`	HTTP 200 OK. Pharmacist Alex authorized for drug stock.	**PASS**	**High**
TC-RBAC-05	RBAC / Lab	Lab Tech Test Access	`GET /api/v1/laboratory/tests/` `Authorization: Bearer`	HTTP 200 OK. Lab Technician David authorized for catalogue.	**PASS**	**High**
TC-RBAC-06	RBAC / Billing	Accountant Invoice Access	`GET /api/v1/billing/invoices/` `Authorization: Bearer`	HTTP 200 OK. Accountant Lisa authorized for financial ledger.	**PASS**	**High**
TC-IDOR-01	Security / IDOR	Cross-Patient Clinical	`GET /api/v1/patients/2/` `Authorization: Bearer`	HTTP 403/404 Forbidden. Patient John cannot inspect Maria.	**PASS**	**Critical**
TC-IDOR-02	Security / IDOR	Cross-Patient Invoice	`GET /api/v1/billing/invoices/1/` `Authorization: Bearer`	HTTP 403/404 Forbidden. Patient John cannot inspect Maria invoice.	**PASS**	**Critical**
TC-SEC-01	Defensive Sec	SQL Injection Vector	`GET /api/v1/patients/?search=' OR '1'='1'` `Admin token`	HTTP 200 OK. ORM parameterized query safely; 0 schema leaks.	**PASS**	**Critical**
TC-SEC-02	Defensive Sec	Reflected XSS Vector	`GET /api/v1/patients/?search=alert(1)`	HTTP 200 OK. Payload safely escaped as literal text string.	**PASS**	**High**
TC-SEC-03	Defensive Sec	Privilege Escalation	`POST /api/v1/accounts/register/` `{'role': 'SUPER_ADMIN', ...}`	HTTP 400 Bad Request. Blocked role elevation on registration.	**PASS**	**Critical**
TC-BILL-01	Billing & Math	Invoice Balance Formula	`Inspect balance_due on Invoices #1 & #2` `Formula: total_amount - paid_amount`	Verified exact mathematical equality (0.00 & 320.00).	**PASS**	**Critical**
TC-INP-01	Inpatient / Wards	Bed Census Consistency	`Inspect Ward W-GEN-01` `Formula: total_beds >= occupied + avail`	Verified consistent (3 total >= 1 occupied + 2 available).	**PASS**	**High**
TC-PHARM-01	Pharmacy Stock	Drug Stock & Safety Alert	`Inspect Amoxicillin, Atorvastatin, Metformin` `Stock count vs low_stock_threshold`	Verified stock valid (250, 180, 320 units; threshold 50).	**PASS**	**Medium**
TC-UI-01	Frontend UI	Dark Mode Toggle	`Click Sun/Moon toggle button in TopNav` `Persist to localStorage key 'hms-theme'`	Verified .dark class added/removed on ; persisted.	**PASS**	**Low**

2. Formal Defect / Bug Report — Complete Technical Audit

Formal defect record logged during QA execution, including root cause analysis, security threat impact, exact code diff remediation, and regression verification.

Bug ID: BUG-001	**Severity:** High	**Priority:** P1 (Urgent)	**Status:** CLOSED (VERIFIED FIXED)
Module: Hospitals / Staff Management (RBAC)	**Environment:** WSGI Dev (127.0.0.1:8000)	**Detected In:** QA Cycle 1 (Test TC-RBAC-01)	**Verified In:** QA Cycle 2 (Post-Remediation)

Defect Title	Patient accounts can programmatically query complete hospital staff roster, employee IDs, and internal designations via GET /api/v1/staff/
Preconditions	1. User account registered with role PATIENT (e.g. patient_john) 2. Valid JWT access token acquired via /api/v1/accounts/login/
Steps to Reproduce	1. Authenticate as patient_john using POST /api/v1/accounts/login/ 2. Send HTTP request: GET http://127.0.0.1:8000/api/v1/staff/ 3. Include Header: Authorization: Bearer <patient_jwt_token> 4. Observe HTTP response status code and data payload.
Expected Result	**HTTP 403 Forbidden**. Patients have no operational need to access internal staff personnel records, employee numbers, joining dates, or internal designations.
Actual Result (Cycle 1)	**HTTP 200 OK**. The API returned complete staff roster with unredacted employee metadata: [{"id": 1, "employee_id": "EMP-NURSE-001", "designation": "Head Inpatient Nurse", ...}, {"id": 2, "employee_id": "EMP-RECEPT-001", ...}]
Root Cause Analysis	In backend/hospitals/views.py, StaffViewSet had: permission_classes = [IsHospitalAdminOrReadOnly]. Django REST Framework's IsHospitalAdminOrReadOnly permits SAFE_METHODS (GET, HEAD, OPTIONS) to **any authenticated caller**. Thus, patient accounts passed the check without restriction.
Remediation Applied	1. Developed custom permission guard IsStaffPersonnel in backend/common/permissions.py which explicitly restricts access to internal staff roles (SUPER_ADMIN, HOSPITAL_ADMIN, DOCTOR, NURSE, RECEPTIONIST, LAB_TECHNICIAN, PHARMACIST, ACCOUNTANT) and explicitly denies PATIENT. 2. Updated StaffViewSet.permission_classes = [IsStaffPersonnel] in backend/hospitals/views.py.
Post-Fix Verification	Re-tested with backend/qa_full_audit.py. The patient request now receives strict **HTTP 403 Forbidden** with payload {"detail": "You do not have permission to perform this action."}. Backend unit test suite re-executed: **198 / 198 passed** with 0 regressions.

Code Remediation Diff Evidence

```
# 1. Created IsStaffPersonnel in backend/common/permissions.py
class IsStaffPersonnel(permissions.BasePermission):
    """Restricts access strictly to internal hospital staff and admin roles; denies PATIENT."""
    STAFF_ROLES = [
        'SUPER_ADMIN', 'HOSPITAL_ADMIN', 'DOCTOR', 'NURSE',
        'RECEPTIONIST', 'LAB_TECHNICIAN', 'PHARMACIST', 'ACCOUNTANT'
    ]
    def has_permission(self, request, view):
        user = request.user
        if not (user and user.is_authenticated):
            return False
        if user.is_superuser or user.role == 'SUPER_ADMIN':
            return True
        return user.role in self.STAFF_ROLES

# 2. Updated backend/hospitals/views.py
class StaffViewSet(viewsets.ModelViewSet):
-   permission_classes = [IsHospitalAdminOrReadOnly]
+   permission_classes = [IsStaffPersonnel]
```

QA AUDITOR SIGN-OFF Lead Software Test & Security Engineer Antigravity QA Core Team	**AUDIT VERDICT** **APPROVED FOR PRODUCTION DEPLOYMENT** All 26 Live Test Cases Passed (100%) • Zero Open Defects	**SECURITY CERTIFICATION** IDOR Protection: VERIFIED SQLi / XSS Defenses: ACTIVE RBAC Boundary Gates: ENFORCED